

SENTINELLE

Cahier des charges — MVP v1.1

Protection des séniors contre les arnaques numériques

Produit : Sentinelle SaaS

Version CDC : MVP v1.1 — Stack mise à jour

Date : Mai 2026

Frontend : Nuxt 3 — déployé sur Vercel

Backend : Adonis 6 — déployé sur VPS
Hetzner

Base de données : PostgreSQL — sur VPS Hetzner

ORM : Lucid ORM (natif Adonis)

1. Contexte et vision produit

1.1 Le problème

En France, 800 000 seniors de plus de 75 ans sont victimes d’escroqueries chaque année. Les fraudes numériques ont bondi de 64 % depuis 2016. En 2025, les arnaques par email exploitées grâce à l’IA (phishing parfait, voix clonée, usurpation d’identité) sont devenues indistinguables des communications légitimes pour une personne âgée.

Les enfants et aidants familiaux ne peuvent pas surveiller en permanence les boîtes mail de leurs parents. Le senior, souvent isolé et peu familier avec les codes numériques, n’a aucun filet de sécurité actif.

1.2 La vision Sentinelle

Sentinelle est une sentinelle silencieuse qui analyse en temps réel les emails entrants d’un senior, évalue leur niveau de risque grâce à l’IA, et alerte l’enfant aidant avant que le senior n’agisse. Le senior ne voit rien changer. L’enfant a la paix de l’esprit.

1.3 Positionnement

Cible principale	L’enfant adulte (35-55 ans) qui a des parents âgés seuls
Cible secondaire	Le senior lui-même, via un guide d’installation simplifié
Canal MVP	Analyse des emails Gmail uniquement
Modèle économique	Abonnement mensuel par adresse email surveillée
Proposition de valeur	« Je sais en temps réel si maman reçoit une arnaque »

2. Architecture technique

2.1 Vue d’ensemble de la stack

Nuxt 3 (Vercel) ↔ API REST → Adonis 6 (Hetzner) ↔ PostgreSQL (Hetzner) + Gmail API + API Anthropic Claude + Stripe + Resend

2.2 Choix et justification de la stack

Brique	Technologie	Justification
Frontend	Nuxt 3 + Tailwind CSS	SSR + SSG, Vue 3 natif, déploiement Vercel sans config, écosystème riche
Hébergement frontend	Vercel	CDN mondial, déploiement auto sur push git, HTTPS automatique
Backend API	Adonis 6 (Node.js)	Framework MVC complet, auth intégrée, ORM Lucid, jobs/queues natifs
Hébergement backend	VPS Hetzner	Contrôle total, coût fixe maîtrisé, même serveur que Tunelab
Base de données	PostgreSQL (Hetzner)	Sur le même VPS, zéro latence avec Adonis, Lucid ORM natif
ORM	Lucid ORM	Migrations Adonis, modèles typisés, relations fluides
Workers	Adonis Jobs (Bouncer)	Jobs asynchrones natifs Adonis pour le polling Gmail
Gmail accès	Google OAuth 2.0 + Gmail API v1	Consentement explicite du senior, lecture seule, révocable
Analyse IA	API Anthropic (claude-sonnet-4-6)	Scoring risque email, JSON structuré, stable et rapide
Paieement	Stripe	Abonnements récurrents, webhooks, portail client self-service
Emails alertes	Resend	API simple, délivrabilité excellente, templates HTML
Process manager	PM2	Gestion des processus Adonis + workers sur Hetzner
Reverse proxy	Nginx	Sur Hetzner : routing, SSL Let's Encrypt, compression

2.3 Architecture Hetzner VPS

Un seul VPS Hetzner (CX21 ou CX31 recommandé) fait tourner : • Nginx (reverse proxy + SSL) • Adonis 6 API (port 3333, géré par PM2) • Adonis Jobs Worker (processus séparé, géré par PM2) • PostgreSQL 16 • pgAdmin (optionnel, port local uniquement)

⚠ Point d'attention : si Tunelab tourne déjà sur ce VPS, vérifier les ressources disponibles. Un CX21 (2 vCPU, 4 GB RAM) suffit pour les deux projets en phase MVP. Prévoir un upgrade vers CX31 si la charge augmente.

2.4 Flux de communication Nuxt ↔ Adonis

1. Le frontend Nuxt (Vercel) appelle l'API REST Adonis via HTTPS (domaine `api.sentinelle.app`)
2. Adonis expose les routes API sous `/api/v1/`

3. L'authentification se fait via JWT (tokens générés par Adonis Auth)
4. Le token JWT est stocké côté client dans un cookie HttpOnly sécurisé
5. Nginx sur Hetzner route les requêtes vers Adonis et gère le SSL

2.5 Flux de traitement d'un email (Worker Adonis)

6. Un job Adonis s'exécute toutes les 5 minutes (scheduler Adonis natif)
7. Pour chaque senior actif : récupère le refresh token OAuth (chiffré en BDD)
8. Interroge Gmail API v1 : messages.list avec filtre is:unread newer_than:5m
9. Pour chaque nouvel email : récupère from, subject, date, body (texte brut, tronqué à 2000 car.)
10. Envoie à l'API Claude avec le prompt système — reçoit le JSON de scoring
11. Stocke le résultat dans PostgreSQL (table email_analyses)
12. Si score >= seuil configuré : envoie alerte email à l'enfant via Resend
13. L'email original n'est PAS modifié dans Gmail

2.6 Sécurité et RGPD

- OAuth Google : consentement explicite du senior, périmètre lecture seule (gmail.readonly)
- Tokens OAuth chiffrés en AES-256 avant stockage PostgreSQL
- Corps des emails NON stockés — uniquement métadonnées + résultat analyse
- Droit à l'oubli : suppression compte = révocation OAuth Google + purge BDD
- Hébergement 100 % EU (Hetzner Allemagne, Vercel EU region)
- HTTPS obligatoire sur toutes les communications Nuxt ↔ Adonis
- Rate limiting sur les routes API Adonis (middleware natif)

3. Parcours utilisateurs

3.1 Deux profils, deux rôles

L'Enfant aidant (Compte payant)	Le Senior (Compte surveillé)
Crée et paye le compte Sentinelle Connecte le Gmail de son parent Configure les seuils et alertes Reçoit les alertes sur son propre email Consulte le dashboard via Nuxt	Autorise l'accès Gmail une seule fois (OAuth) N'utilise pas l'interface Sentinelle Continue d'utiliser Gmail normalement Peut révoquer l'accès à tout moment

3.2 Mode A — Installation par l'enfant (recommandé)

14. L'enfant s'inscrit sur sentinelle.app (Nuxt), crée son compte via l'API Adonis
15. Il saisit le prénom et l'adresse Gmail de son parent
16. Adonis génère un lien OAuth unique et l'envoie au senior par email (Resend)

17. L'enfant ouvre ce lien SUR l'appareil du senior et clique « Autoriser »
18. Google retourne le code → Adonis l'échange contre un refresh token → chiffré en BDD
19. Surveillance active. L'enfant reçoit une confirmation par email.

3.3 Mode B — Autonome senior

20. Le senior reçoit un email avec un bouton unique « Protéger mon Gmail »
21. Page Nuxt ultra-simple : explication en 3 lignes + un seul bouton
22. Redirection vers consentement Google OAuth (page Google standard)
23. Confirmation : « Votre boîte mail est maintenant protégée »
24. L'enfant est notifié de l'activation automatiquement

4. Fonctionnalités détaillées

ID	Fonctionnalité	Description	Priorité	Complexité
F01	Inscription enfant	Formulaire Nuxt → API Adonis → création compte + JWT	MVP	Faible
F02	Ajout senior	Formulaire prénom + Gmail, génération lien OAuth unique	MVP	Faible
F03	Connexion Gmail OAuth	Flow OAuth Google, échange code → token, chiffrement AES-256	MVP	Moyenne
F04	Worker polling Gmail	Job Adonis toutes les 5 min, Gmail API v1, déduplication	MVP	Moyenne
F05	Analyse IA Claude	Appel API Anthropic, retour JSON score/catégorie/explication	MVP	Faible
F06	Seuil alerte configurable	L'enfant règle le seuil 0-100 (défaut 60) via dashboard Nuxt	MVP	Faible
F07	Alerte email Resend	Email structuré à l'enfant : objet, score visuel, explication, conseils	MVP	Faible
F08	Dashboard Nuxt	Liste des emails analysés, tri par risque, code couleur	MVP	Moyenne
F09	Détail analyse	Vue détail email : score, catégorie, explication IA, conseils à donner	MVP	Faible
F10	Abonnement Stripe	Paieement mensuel, gestion via webhook Adonis, activation auto	MVP	Moyenne
F11	Pause surveillance	Toggle actif/pause par senior, stoppe le worker pour ce compte	MVP	Faible
F12	Révocation OAuth	Déconnexion propre : révocation Google + suppression token BDD	MVP	Faible
F13	Push notifications	PWA ou service push via Nuxt pour alertes mobile en temps réel	V2	Moyenne
F14	Résumé hebdo senior	Email optionnel senior : « X alertes cette semaine »	V2	Faible

ID	Fonctionnalité	Description	Priorité	Complexité
F15	Multi-seniors	Un compte enfant → plusieurs adresses surveillées	V2	Faible
F16	Expéditeurs de confiance	Liste blanche : certains expéditeurs toujours ignorés	V2	Moyenne
F17	Rapport mensuel PDF	Bilan des menaces, généré par Adonis, envoyé via Resend	V3	Moyenne
F18	Extension SMS Android	Surveillance SMS via app Android native (scope élargi)	V3	Elevée

5. Prompt système — API Anthropic Claude

Ce prompt est injecté dans chaque appel API depuis le worker Adonis. Il est conçu pour retourner un JSON strict, directement désérialisable par Lucid/TypeScript sans post-traitement.

SYSTEM PROMPT :Tu es Sentinelle, un analyseur expert en détection d'arnaques par email ciblant les personnes âgées en France.Retourne UNIQUEMENT un objet JSON valide, sans texte avant ni après, sans balises markdown.FORMAT STRICT :{ "score": [0 à 100], "categorie": "[voir liste ci-dessous]", "niveau": "[bas | moyen | eleve | critique]", "explication": "[1 phrase max, français simple, compréhensible par un non-technicien]", "conseil_enfant": "[action concrète à faire ou dire au senior]", "indicateurs": ["signal1", "signal2"]}CATEGORIES : legitime | phishing | arnaque_financiere | usurpation_identite |urgence_familiale | démarchage_abusif | malware | spam | inconnuSCORE : 0-20 sûr | 21-40 faible risque | 41-60 modéré | 61-80 élevé | 81-100 critiqueINDICES À ANALYSER : urgence exagérée, demande d'infos bancaires, liens suspects,usurpation organisme officiel (CPAM/impôts/banque/police), promesses irréalistes,demande de virement urgent, fausse détresse d'un proche, pression temporelle.

USER MESSAGE (injecté dynamiquement par le worker Adonis) :Expéditeur : {from}Objet : {subject}Date : {date}Corps (2000 premiers caractères max) :{body_text_truncated}Analyse cet email.

Paramètres API recommandés : model: claude-sonnet-4-6 | max_tokens: 300 | temperature: 0Avec temperature: 0, les réponses sont déterministes — essentiel pour un scoring stable.

6. Schéma base de données PostgreSQL + Lucid

Migrations Adonis — créer avec : node ace make:migration nom_migration

Table	Colonne	Type / Contrainte / Description
users	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()
users	email	varchar(255) UNIQUE NOT NULL
users	password	varchar(255) NOT NULL — hashé par Adonis Auth
users	remember_me_token	varchar(255) NULL
users	created_at	timestamp NOT NULL
users	updated_at	timestamp NOT NULL
seniors	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()
seniors	user_id	uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE
seniors	prenom	varchar(100) NOT NULL
seniors	gmail	varchar(255) NOT NULL
seniors	oauth_access_token	text NULL — chiffré AES-256
seniors	oauth_refresh_token	text NULL — chiffré AES-256
seniors	oauth_expiry	timestamp NULL
seniors	actif	boolean NOT NULL DEFAULT true
seniors	seuil_alerte	smallint NOT NULL DEFAULT 60
seniors	last_polled_at	timestamp NULL
seniors	created_at	timestamp NOT NULL
seniors	updated_at	timestamp NOT NULL
email_analyses	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()
email_analyses	senior_id	uuid NOT NULL REFERENCES seniors(id) ON DELETE CASCADE
email_analyses	gmail_message_id	varchar(255) NOT NULL UNIQUE
email_analyses	from_email	varchar(255) NOT NULL
email_analyses	subject	text NOT NULL
email_analyses	date_received	timestamp NOT NULL
email_analyses	score	smallint NOT NULL
email_analyses	categorie	varchar(50) NOT NULL
email_analyses	niveau	varchar(20) NOT NULL
email_analyses	explication	text NOT NULL
email_analyses	conseil_enfant	text NOT NULL
email_analyses	indicateurs	jsonb NULL
email_analyses	alerte_envoyee	boolean NOT NULL DEFAULT false
email_analyses	created_at	timestamp NOT NULL
subscriptions	id	uuid PRIMARY KEY DEFAULT gen_random_uuid()
subscriptions	user_id	uuid NOT NULL REFERENCES users(id)
subscriptions	stripe_customer_id	varchar(255) UNIQUE NOT NULL
subscriptions	stripe_subscription_id	varchar(255) UNIQUE NULL
subscriptions	plan	varchar(50) NOT NULL — solo famille

Table	Colonne	Type / Contrainte / Description
subscriptions	statut	varchar(50) NOT NULL — active paused cancelled
subscriptions	current_period_end	timestamp NULL
subscriptions	created_at	timestamp NOT NULL
subscriptions	updated_at	timestamp NOT NULL

7. Modèle économique

7.1 Tarification

Plan	Prix	Inclus
Solo Senior	7,90 € / mois	1 adresse Gmail surveillée, alertes email illimitées
Famille (recommandé)	12,90 € / mois	Jusqu'à 3 adresses (parents + beaux-parents)

7.2 Estimation des coûts opérationnels

Coût API Claude / senior	~600 emails/mois × 0,003 € = ~1,80 €/mois
Coût VPS Hetzner	~8-15 €/mois (partagé avec Tunelab)
Vercel (frontend)	Gratuit jusqu'à 100 Go bande passante/mois
Resend (emails)	Gratuit jusqu'à 3 000 emails/mois, puis 1 €/1 000
Marge brute estimée	~5,50 €/client/mois au plan Solo
Seuil de rentabilité	~25-30 abonnements actifs
Objectif 6 mois	100 abonnements = ~790 € MRR

8. Roadmap de développement

Phase	Durée	Livraisons
Phase 1 — Fondations	2 semaines	Init Adonis 6 + PostgreSQL Hetzner, Lucid migrations, Auth JWT, init Nuxt 3 Vercel, pipeline CI/CD git
Phase 2 — OAuth Gmail	1-2 semaines	Flow OAuth Google complet, chiffrement AES-256 tokens, stockage BDD, page activation senior simplifiée
Phase 3 — Worker + IA	1-2 semaines	Job Adonis scheduler, Gmail API polling, appel API Claude, stockage résultats, gestion erreurs et rate limiting
Phase 4 — Alertes + Dashboard	1 semaine	Système alertes Resend, dashboard Nuxt (liste + détail), configuration seuil, code couleur risque
Phase 5 — Stripe + Beta	1 semaine	Intégration Stripe, webhooks Adonis, activation/pause, tests complets, beta privée 5-10 familles
V2	4-6 semaines	Push notifications PWA, résumé hebdo senior, multi-seniors, liste blanche expéditeurs, historique 30j
V3	TBD	App Android SMS, rapport PDF mensuel, tableau de bord analytics

9. Instructions pour Claude Code

Cette section est rédigée directement à l'attention de Claude Code.

9.1 Initialisation du projet

Backend Adonis (sur Hetzner)

- `npm init adonis-ts-app@latest sentinelle-api --kit=api`
- Configurer la connexion PostgreSQL dans `config/database.ts` (Lucid)
- Installer : `@adonisjs/auth`, `@adonisjs/lucid`, `@google-cloud/local-auth`, `googleapis`, `@anthropic-ai/sdk`, `stripe`, `resend`
- Configurer PM2 pour gérer les processus Adonis + worker séparément
- Configurer Nginx : `proxy_pass` vers `localhost:3333`, SSL Let's Encrypt

Frontend Nuxt (sur Vercel)

- `npx nuxi@latest init sentinelle-front`
- Installer : `@nuxtjs/tailwindcss`, `pinia`, `@vueuse/nuxt`
- Configurer `NUXT_PUBLIC_API_URL` vers `https://api.sentinelle.app`
- Connecter le dépôt Git à Vercel pour déploiement automatique

9.2 Variables d'environnement — Backend Adonis (.env)

```
# Base de données
PostgreSQLDB_CONNECTION=pgPG_HOST=localhostPG_PORT=5432PG_DB_NAME=s
entinellePG_USER=sentinelle_userPG_PASSWORD=# AdonisAPP_KEY=#
node ace generate:keyHOST=0.0.0.0PORT=3333NODE_ENV=production# Google
OAuthGOOGLE_CLIENT_ID=GOOGLE_CLIENT_SECRET=GOOGLE_REDIRECT_URI=ht
tps://api.sentinelle.app/auth/google/callback# Chiffrement tokens OAuth (clé 32 bytes en
hex)ENCRYPTION_KEY=# AnthropicANTHROPIC_API_KEY=#
StripeSTRIPE_SECRET_KEY=STRIPE_WEBHOOK_SECRET=#
ResendRESEND_API_KEY=EMAIL_FROM=alertes@sentinelle.app# Frontend URL (pour
CORS)FRONTEND_URL=https://sentinelle.app
```

9.3 Points d'attention critiques pour Claude Code

- Chiffrer les tokens OAuth AVANT toute écriture en BDD (AES-256-GCM, IV unique par token)
- Le worker Gmail doit stocker le gmail_message_id pour éviter les doublons (contrainte UNIQUE en BDD)
- Utiliser le scope Gmail le plus restrictif possible : <https://www.googleapis.com/auth/gmail.readonly>
- Tronquer le body email à 2000 caractères avant l'appel API Claude (contrôle des coûts)
- Implémenter un try/catch sur chaque appel API Claude : si échec, logger sans alerter l'utilisateur
- Les webhooks Stripe doivent être vérifiés avec la signature Stripe (stripe.webhooks.constructEvent)
- Le dashboard Nuxt doit être accessible uniquement après authentification (middleware Nuxt auth)
- Interface design : vert (#0F6E56) pour email sûr, orange (#BA7517) pour attention, rouge (#A32D2D) pour danger
- Ne jamais afficher le corps complet de l'email dans le dashboard — uniquement l'objet et l'analyse